

POLINES · ADMINISTRASI BISNIS · ABT-3D

Lembar Kerja Minggu 6

Basis Data SQLite dan Operasi Data Dasar (CRUD)

Pemrograman Bisnis & Perancangan Sistem Administrasi · Senin 90 menit: membangun database. Selasa 90 menit: CRUD dengan aplikasi Streamlit.

Nama		NIM	
Tema proyek		Nama file .db	

Petunjuk. Ikuti langkah sesuai urutan dan ketik kode persis seperti contoh. Setiap tahap punya kotak CEK: centang bila hasil Anda sama sebelum lanjut. Pakai data simulasi, jangan data pribadi pelanggan nyata. File akhir: kasir.db, buat_database.py, uji_aturan.py, database.py, app.py, dan lembar ini terisi.

o. Persiapan

1. Buka VS Code, pilih **File → Open Folder...**, lalu buka folder proyek Minggu 5 (yang berisi `main.py` dan `logika_bisnis.py`).
2. Buat folder baru minggu-06 di dalamnya agar file Minggu 5 tidak tertimpa. Buka folder minggu-06 itu di VS Code.
3. Pilih **Terminal → New Terminal**. Ketik `python --version` (Windows: `py --version` bila perlu). Harus tampil Python 3.
4. Senin tidak perlu memasang apa pun: modul `sqlite3` sudah bawaan Python. **Sebelum Selasa**, pasang Streamlit satu kali: `python -m pip install streamlit` (perlu internet; lewati bila sudah terpasang sejak Minggu 5).

CEK ☐ Terminal berada di folder minggu-06 dan menampilkan versi Python 3.

BAGIAN A · SENIN — Dari rancangan ke tabel yang hidup

Bekal ERD 1/3 · Peta data, bukan alur program

Baca bekal ini pada kegiatan A1. Seluruh penjelasan tersedia di Minggu 6; Anda tidak perlu membuka modul Minggu 3.

ERD (Entity Relationship Diagram) adalah gambar rancangan data: apa yang dicatat, informasi apa yang disimpan, dan bagaimana catatan saling terhubung. Seperti merancang buku daftar produk dan buku penjualan sebelum mengisinya.

ERD bukan diagram alur. ERD menunjukkan struktur dan hubungan data, bukan urutan klik, menu, atau langkah program.

Istilah	Arti mudah	Contoh kasir
Entitas	Jenis objek/kejadian yang dicatat; menjadi tabel.	produk dan transaksi
Atribut	Keterangan yang disimpan; menjadi kolom.	nama, harga, stok
Baris/record	Satu catatan nyata di tabel.	Buku Tulis, harga 5000, stok 20
Relasi	Hubungan antarentitas.	Produk tercatat dalam transaksi

Entitas ≠ isi baris. “produk” adalah entitas; “Buku Tulis” adalah salah satu baris di dalamnya. Jangan membuat tabel baru untuk setiap nama barang.

PK = nomor identitas catatan

Primary Key (PK) membedakan setiap baris: harus unik dan tidak kosong. `produk.id_produk = 1` berarti satu produk tertentu. Nama bukan pilihan PK yang aman karena nama dapat sama atau berubah.

FK = nomor rujukan ke catatan lain

Foreign Key (FK) menghubungkan tabel. `transaksi.id_produk` merujuk `produk.id_produk`. FK boleh berulang karena produk yang sama dapat dibeli berkali-kali. Pada contoh ini FK wajib terisi (NOT NULL).

Nama kolom sama, peran berbeda: `id_produk` adalah PK di produk, tetapi FK di transaksi.

Bekal ERD 2/3 · Cara membaca hubungan

Kardinalitas menjawab: satu catatan berhubungan dengan berapa catatan di tabel lain? Hubungan ditentukan oleh aturan bisnis, bukan jumlah baris yang kebetulan sudah diisi.

Hubungan	Arti / contoh
1:1	Satu pegawai memiliki paling banyak satu kartu akses aktif; satu kartu milik satu pegawai.
1:N	Satu produk dapat muncul di banyak catatan transaksi; setiap catatan transaksi menunjuk satu produk.
M:N	Satu nota memuat banyak produk; satu produk muncul di banyak nota. Perlu tabel penghubung detail_transaksi.

Contoh ERD praktik minggu ini: hubungan 1:N

PRODUK	TRANSAKSI
PK id_produk nama harga stok	PK id_transaksi tanggal FK id_produk jumlah harga_satuan

produk.id_produk (PK) ← transaksi.id_produk (FK)

Satu produk → 0..N transaksi. Setiap transaksi → tepat 1 produk.

1 = satu; N = banyak; 0..N = boleh belum terjual atau terjual berkali-kali. FK berada di transaksi, yaitu sisi banyak.

Produk yang belum terjual tidak perlu dibuatkan transaksi kosong.

produk.id_produk (PK)	nama
1	Buku Tulis
2	Pulpen

Contoh data untuk membaca hubungan (bukan data awal tambahan):

id_transaksi (PK)	id_produk (FK)	jumlah
1	1	2
2	1	1

Dua transaksi menunjuk Buku Tulis ID 1: sah, FK boleh berulang. Pulpen ID 2 belum mempunyai transaksi: juga sah. FK 999 ditolak jika produk 999 tidak ada dan penjaga FK aktif.

Batas contoh: transaksi pada skrip ini mencatat penjualan satu jenis produk, bukan nota lengkap. Nota banyak produk memerlukan transaksi (kepala nota) + detail_transaksi (baris barang) + produk. Struktur lanjutan itu tidak wajib dibuat pada praktik dasar ini.

Bekal ERD 3/3 · Dari gambar menjadi tabel SQLite

Rancangan ERD	Terjemahan SQLite
Entitas produk	CREATE TABLE produk (...)
Atribut nama, harga, stok	nama TEXT; harga INTEGER; stok INTEGER
PK produk.id_produk	id_produk INTEGER PRIMARY KEY AUTOINCREMENT
FK transaksi.id_produk	id_produk INTEGER NOT NULL + FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
Aturan isian	NOT NULL untuk wajib terisi; CHECK untuk nilai yang diperbolehkan

Urutan merancang

1. Sebutkan dua entitas dan tulis hubungan dengan satu kalimat bisnis.
2. Tentukan atribut dan PK pada setiap entitas.
3. Untuk 1:N, letakkan FK di sisi banyak. ID PK dan FK contoh ini sama-sama INTEGER.
4. Tulis FOREIGN KEY ... REFERENCES ... untuk menyatakan rujukan.
5. Tentukan aturan isian. Harga rupiah dan stok = INTEGER; tanggal = TEXT berformat YYYY-MM-DD.
6. Saat memakai SQLite, aktifkan PRAGMA foreign_keys = ON di setiap koneksi.

Aturan isian melengkapi ERD, bukan menggantikan hubungan. NOT NULL melarang nilai kosong (NULL); CHECK menjaga nilai, misalnya harga > 0. SQLite perlu PRAGMA agar aturan FK benar-benar dijalankan.

Cek pemahaman sebelum mengetik

1. “Buku Tulis” itu entitas atau isi satu baris? Mengapa?

Jawaban:

2. Mengapa transaksi.id_produk boleh berulang, tetapi produk.id_produk tidak?

Jawaban:

3. Di tabel mana FK diletakkan untuk hubungan produk–transaksi?

Jawaban:

4. Apakah produk boleh belum memiliki transaksi? Jelaskan arti 0..N.

Jawaban:

Lanjutkan A1: pakai contoh produk–transaksi di atas. Untuk proyek lain, buat rancangan dengan pola yang sama; bahan lama tidak menjadi prasyarat.

A1. Pahami ERD dan rancang tabel menit 10–25

Gunakan teori dan contoh ERD di bagian ini; tidak perlu membuka modul Minggu 3. Isi tabel di bawah untuk produk dan transaksi, lalu rancang satu entitas tambahan untuk tema proyek Anda. Baris pertama adalah contoh.

Tabel	Primary Key	Kolom + tipe data	Foreign Key ke	Aturan (CHECK)
transaksi	id_transaksi	tanggal TEXT, id_produk INTEGER, jumlah INTEGER, harga_satuan INTEGER	produk	jumlah > 0

Ingat. Rupiah disimpan sebagai INTEGER (angka bulat). Tanggal disimpan sebagai TEXT berformat 'YYYY-MM-DD'.

A2. Buat database dengan buat_database.py menit 25–40

1. Salin file `buat_database.py` dari dosen ke folder minggu-06, atau buat file baru dan ketik kode berikut.
2. Baca komentar di kode (tulisan setelah tanda # atau --). Komentar tidak dijalankan Python.
3. Simpan (Ctrl+S). Di terminal ketik `python buat_database.py` lalu Enter.
4. Jalankan sekali lagi. Tidak boleh ada error, dan data awal tidak bertambah dua kali.

```

buat_database.py
import sqlite3

koneksi = sqlite3.connect("kasir.db")
koneksi.execute("PRAGMA foreign_keys = ON") # wajib: di SQLite FK mati secara bawaan

koneksi.executescript("""
CREATE TABLE IF NOT EXISTS produk (
    id_produk    INTEGER PRIMARY KEY AUTOINCREMENT,
    nama         TEXT     NOT NULL,
    harga        INTEGER NOT NULL CHECK (typeof(harga) = 'integer' AND harga > 0),
    stok         INTEGER NOT NULL DEFAULT 0 CHECK (stok >= 0)
);

CREATE TABLE IF NOT EXISTS transaksi (
    id_transaksi INTEGER PRIMARY KEY AUTOINCREMENT,
    tanggal       TEXT     NOT NULL,           -- format 'YYYY-MM-DD'
    id_produk     INTEGER NOT NULL,
    jumlah        INTEGER NOT NULL CHECK (jumlah > 0),
    harga_satuan  INTEGER NOT NULL,           -- harga saat transaksi terjadi
    FOREIGN KEY (id_produk) REFERENCES produk(id_produk)
);
""")

# Data awal hanya diisi bila tabel produk masih kosong,
# sehingga skrip aman dijalankan berkali-kali.
jumlah_produk = koneksi.execute("SELECT COUNT(*) FROM produk").fetchone()[0]
if jumlah_produk == 0:
    koneksi.executemany(

```

```

        "INSERT INTO produk (nama, harga, stok) VALUES (?, ?, ?)",
        [
            ("Buku Tulis", 5000, 20),
            ("Pulpen", 3000, 30),
            ("Map Plastik", 2500, 10),
        ],
    )
    koneksi.execute(
        "INSERT INTO transaksi (tanggal, id_produk, jumlah, harga_satuan) "
        "VALUES (?, ?, ?, ?)",
        ("2026-10-12", 1, 2, 5000),
    )
    koneksi.commit()

koneksi.close()
print("Database dan tabel siap.")

```

Bagian kode	Artinya
PRAGMA foreign_keys = ON	Menyalakan penjaga relasi. Di SQLite penjaga ini mati secara bawaan, jadi wajib ditulis di setiap koneksi.
IF NOT EXISTS	Buat tabel hanya bila belum ada, sehingga skrip aman dijalankan berkali-kali.
CHECK (...)	Aturan isian. Harga harus angka bulat lebih dari nol; stok tidak boleh negatif.
FOREIGN KEY	Setiap transaksi wajib merujuk produk yang benar-benar ada.
harga_satuan	Harga pada hari transaksi, seperti nota. Transaksi lama tidak berubah walau harga produk naik.

CEK ☐ Terminal menulis “Database dan tabel siap.” dan file kasir.db muncul di panel Explorer.

A3. Tambahkan satu tabel dari rancangan A1 menit 40–65

1. Pilih satu entitas tambahan yang Anda rancang di A1 (misalnya pemasok). Tentukan atribut, PK, dan hubungannya dengan tabel lain menggunakan panduan ERD di bagian ini.
2. Tulis perintah `CREATE TABLE IF NOT EXISTS ...` di kotak berikut, lalu tambahkan ke dalam tanda `"""` pada `buat_database.py`.
3. Bila tabel itu berelasi, jangan lupa kolom FK dan baris `FOREIGN KEY ... REFERENCES ...`.
4. Jalankan lagi `python buat_database.py`. Tema selain kasir boleh mengganti nama file menjadi `<nama_proyek>.db` di semua skrip.

Tulis `CREATE TABLE` tabel tambahan Anda di sini.

CEK ☐ Skrip berjalan tanpa error setelah tabel tambahan dimasukkan.

A4. Uji penjaga database menit 65–85

Buat file `uji_aturan.py`, ketik kode berikut, lalu jalankan dengan `python uji_aturan.py`. Skrip ini mencoba memasukkan data yang melanggar aturan, lalu membatalkannya (rollback) agar database tetap bersih.

```
uji_aturan.py
import sqlite3

def coba(judul, koneksi, perintah_sql):
    try:
        koneksi.execute(perintah_sql)
        print(judul, "-> DITERIMA")
    except sqlite3.IntegrityError as e:
        print(judul, "-> DITOLAK:", e)

# Koneksi dengan penjaga relasi aktif
koneksi = sqlite3.connect("kasir.db")
koneksi.execute("PRAGMA foreign_keys = ON")
coba("A. Harga berupa teks", koneksi,
      "INSERT INTO produk (nama, harga) VALUES ('Pensil', 'seribu')")
coba("B. Produk tidak ada", koneksi,
      "INSERT INTO transaksi (tanggal, id_produk, jumlah, harga_satuan) "
      "VALUES ('2026-10-12', 999, 1, 5000)")
koneksi.rollback() # batalkan apa pun yang sempat masuk
koneksi.close()

# Koneksi TANPA baris PRAGMA: lihat bedanya
koneksi = sqlite3.connect("kasir.db")
coba("C. Produk tidak ada, tanpa PRAGMA", koneksi,
      "INSERT INTO transaksi (tanggal, id_produk, jumlah, harga_satuan) "
      "VALUES ('2026-10-12', 999, 1, 5000)")
koneksi.rollback() # jangan simpan data yatim hasil uji C
koneksi.close()
```

Uji	Yang dicoba	Hasil yang diharapkan	Hasil saya (salin dari terminal)
A	Harga diisi teks 'seribu'	DITOLAK: CHECK constraint failed	
B	Transaksi untuk produk ID 999	DITOLAK: FOREIGN KEY constraint failed	
C	Sama seperti B, tanpa PRAGMA	DITERIMA	

Pertanyaan: mengapa uji C diterima padahal datanya sama dengan uji B? Apa akibatnya bagi laporan bila hal ini terjadi pada data nyata?

A5. Refleksi Senin

Tanpa aturan CHECK, SQLite mau menyimpan teks “seribu” di kolom angka. Siapa yang sebaiknya menjaga pintu masuk data, program Python atau database? Jelaskan dengan kata-kata Anda.

CEK ☐ kasir.db, buat_database.py, dan uji_aturan.py tersimpan di folder minggu-06 untuk dipakai hari Selasa, dan Streamlit sudah terpasang.

BAGIAN B · SELASA — CRUD lewat aplikasi Streamlit

Hari ini data dikelola lewat **halaman web Streamlit**. Tampilan dan penyimpanan dipisah ke file yang berbeda: app.py hanya memanggil fungsi di database.py. Kelas yang belum dapat memasang Streamlit memakai jalur cadangan menu terminal main.py sesuai arahan dosen; database.py tetap sama.

File	Tugas
app.py	BARU: tampilan Streamlit — formulir, tombol, pesan, dan tabel produk
database.py	BARU: menyimpan dan mengambil data dari kasir.db. Tidak berisi kode Streamlit.
logika_bisnis.py	Perhitungan dari Minggu 5 (subtotal, diskon). Tidak diubah minggu ini.

B1. Pahami database.py menit 10–20

1. Salin database.py dan app.py dari dosen ke folder minggu-06, satu folder dengan kasir.db.
2. Baca database.py. Perhatikan pola yang selalu sama: buka koneksi → try → execute → commit → finally close.
3. Perhatikan tanda tanya ?. Nilai dikirim sebagai parameter, tidak dirangkai ke teks SQL. Ini mencegah error karena tanda kutip pada nama (misalnya Toko D'Arso) dan mencegah isian jahat.

database.py (bagian yang sudah jadi)

```
import sqlite3

NAMA_DB = "kasir.db"

def buka_koneksi():
    koneksi = sqlite3.connect(NAMA_DB)
    koneksi.execute("PRAGMA foreign_keys = ON") # ulangi di setiap koneksi
    return koneksi

def tambah_produk(nama, harga, stok):
    koneksi = buka_koneksi()
    try:
        koneksi.execute(
            "INSERT INTO produk (nama, harga, stok) VALUES (?, ?, ?)",
            (nama, harga, stok),
        )
        koneksi.commit()
    finally:
        koneksi.close() # tetap ditutup walau terjadi error

def daftar_produk():
    koneksi = buka_koneksi()
    try:
        return koneksi.execute(
            "SELECT id_produk, nama, harga, stok FROM produk"
        ).fetchall()
    finally:
        koneksi.close()
```

```
def ubah_harga(id_produk, harga_baru):
    koneksi = buka_koneksi()
    try:
        cursor = koneksi.execute(
            "UPDATE produk SET harga = ? WHERE id_produk = ?",
            (harga_baru, id_produk),
        )
        koneksi.commit()
        return cursor.rowcount    # 0 berarti ID tidak ditemukan
    finally:
        koneksi.close()
```

Mengapa try / finally? Bagian finally selalu dijalankan walaupun terjadi error, sehingga koneksi pasti ditutup dan file .db tidak terkunci. Catatan: with `sqlite3.connect(...)` hanya mengurus commit, tidak menutup koneksi.

B2. Lengkapi fungsi `hapus_produk` menit 20–35

Fungsi `hapus_produk(id_produk)` di bagian bawah `database.py` masih berisi `raise NotImplementedError`. Ganti isinya dengan pola seperti `ubah_harga`. Petunjuk: perintah SQL-nya `DELETE FROM produk WHERE id_produk = ?`, parameternya `(id_produk,)` (perhatikan koma), dan fungsi mengembalikan `cursor.rowcount`.

Tulis fungsi `hapus_produk` di sini, lalu salin ke `database.py`.

Awat! DELETE atau UPDATE tanpa WHERE akan mengenai SEMUA baris di tabel.

B3. Jalankan dan lengkapi `app.py` (Streamlit) menit 35–70

1. Periksa Streamlit: `python -m streamlit version`. Bila belum ada, jalankan `python -m pip install streamlit`.
2. Jalankan aplikasi: `python -m streamlit run app.py`. Browser membuka halaman **Kelola Produk**. Bila tidak terbuka, salin alamat `http://localhost:8501` dari terminal.
3. Coba tambah satu produk, lalu lihat tabel **Daftar produk** di bagian bawah. Tombol ubah harga dan hapus masih menampilkan pesan “belum dilengkapi”.
4. Untuk menghentikan aplikasi, klik terminal lalu tekan **Ctrl+C**.

```
app.py (bagian yang sudah jadi)
import sqlite3

import streamlit as st

import database

st.title("Kelola Produk")
```

```
# ----- CREATE: tambah produk (sudah jadi) -----
st.subheader("Tambah produk")
with st.form("form_tambah", clear_on_submit=True):
    nama = st.text_input("Nama produk")
    harga = st.number_input("Harga (Rp, angka bulat)", step=1, value=0)
    stok = st.number_input("Stok awal", min_value=0, step=1, value=0)
    tombol_tambah = st.form_submit_button("Simpan produk")

if tombol_tambah:
    if nama.strip() == "":
        st.error("Nama produk wajib diisi.") # validasi Python
    else:
        try:
            database.tambah_produk(nama.strip(), harga, stok)
            st.success(f"Produk {nama.strip()} tersimpan.")
        except sqlite3.IntegrityError as e:
            st.error(f"Database menolak data: {e}") # aturan database

# ... bagian ubah harga (Latihan E1) dan hapus (Latihan E2) ...

# ----- READ: daftar produk (sudah jadi, paling bawah) -----
st.subheader("Daftar produk")
st.dataframe(
    [
        {"ID": id_produk, "Nama": nama, "Harga": harga, "Stok": stok}
        for id_produk, nama, harga, stok in database.daftar_produk()
    ],
    hide_index=True,
)
```

Cara kerja Streamlit. Setiap kali tombol ditekan, **seluruh** `app.py` dijalankan ulang dari atas. `st.form` mengumpulkan isian sampai tombol formulir ditekan. `st.number_input(step=1)` hanya menerima angka bulat, jadi isian abc tidak bisa diketik. Tabel diletakkan paling bawah agar dibaca setelah perubahan diproses.

Latihan E1 – Ubah harga. Ganti baris `st.info(...)` pada bagian ubah harga. Panggil `database.ubah_harga(id_ubah, harga_baru)`: hasil 0 → `st.warning` (ID tidak ditemukan), selainnya → `st.success`. Tangkap `sqlite3.IntegrityError` dengan `st.error`.

Tulis kode pengganti `st.info` pada bagian ubah harga di sini, lalu salin ke `app.py`.

Latihan E2 – Hapus. Ganti baris `st.info(...)` pada bagian hapus. Jika kotak konfirmasi yakin belum dicentang → `st.warning` dan **jangan** menghapus apa pun. Jika sudah dicentang → panggil `database.hapus_produk(id_hapus)`, tangani hasil 0 dan `sqlite3.IntegrityError`.

Tulis kode pengganti `st.info` pada bagian hapus di sini, lalu salin ke `app.py`.

Simpan `app.py`, lalu klik **Rerun** di pojok kanan atas halaman atau tekan **R** agar perubahan kode terbaca.

CEK ☐ Halaman Kelola Produk menampilkan tiga produk data awal; ubah harga dan hapus tidak lagi menampilkan pesan “belum dilengkapi”.

B4. Uji silang delapan skenario menit 70–85

Jalankan setiap skenario berurutan di halaman Streamlit. Setelah setiap aksi, periksa tabel **Daftar produk** di bawah halaman. Tulis hasil yang benar-benar Anda lihat, dan ambil tangkapan layar skenario 1, 4a, dan 6.

No	Skenario & langkah	Hasil yang diharapkan	Hasil saya	Sesuai?
1	Tambah: Penghapus, 2000, stok 15	Pesan hijau “Produk Penghapus tersimpan.”; muncul dengan ID baru		☐ Ya ☐ Tidak
2	Ubah harga ID 2 (Pulpen) menjadi 3500	Pesan berhasil; harga Pulpen 3500 di tabel		☐ Ya ☐ Tidak
3	Ubah harga ID 999	Pesan kuning “ID 999 tidak ditemukan.”		☐ Ya ☐ Tidak
4a	Hapus ID 1 (Buku Tulis, sudah punya transaksi), konfirmasi dicentang	Pesan merah: FOREIGN KEY constraint failed; data utuh		☐ Ya ☐ Tidak
4b	Hapus ID 3 (Map Plastik), konfirmasi dicentang	Pesan berhasil; produk lain utuh		☐ Ya ☐ Tidak
5	Tambah dengan nama kosong / hanya spasi	“Nama produk wajib diisi.” (validasi Python)		☐ Ya ☐ Tidak
6	Tambah dengan harga -5	Pesan merah: CHECK constraint failed		☐ Ya ☐ Tidak
7	Hapus ID 2 tanpa mencentang konfirmasi	Pesan peringatan; tidak ada yang terhapus		☐ Ya ☐ Tidak
8	Hentikan aplikasi (Ctrl+C), jalankan lagi	Hasil skenario 1–4b tetap ada di tabel		☐ Ya ☐ Tidak

Skenario 5 ditolak oleh **Python** (lapis depan); skenario 6 ditolak oleh **database** (lapis belakang). Kolom harga sengaja tidak diberi batas minimum agar aturan CHECK dapat dibuktikan.

Satu perbaikan yang saya lakukan selama pengujian:

B5. Refleksi dan jembatan ke Minggu 7

Minggu 7 akan menambah fitur catat penjualan dan laporan di aplikasi Streamlit. Bagian mana dari pekerjaan hari ini yang tidak perlu diubah sama sekali, dan mengapa? Pada skenario 5 dan 6, lapis mana yang menolak data dan mengapa keduanya diperlukan?

Kotak bantuan: pesan error yang sering muncul

Pesan di terminal / browser	Kemungkinan penyebab dan solusi
no such table: produk	kasir.db dibuat di folder lain atau buat_database.py belum dijalankan. Pastikan terminal di folder yang sama, lalu jalankan buat_database.py.
No module named streamlit	Streamlit belum terpasang. Jalankan python -m pip install streamlit dengan perintah Python yang sama (python atau py).
'streamlit' is not recognized	Gunakan python -m streamlit run app.py, bukan streamlit run app.py.
NotImplementedError: hapus_produk()	Latihan B2 belum dikerjakan. Lengkapi hapus_produk() di database.py.
database is locked	Ada program lain yang masih membuka kasir.db (misalnya SQLite Viewer). Tutup lalu coba lagi.
No module named 'database'	database.py tidak satu folder dengan app.py, atau nama file salah ketik (database.py.txt).
IntegrityError: ... constraint failed	Bukan bug: database menolak data yang melanggar aturan. Baca nama aturan yang disebut.
IndentationError	Jumlah spasi di awal baris tidak konsisten. Gunakan 4 spasi per tingkat.

Daftar periksa pengumpulan

☐	kasir.db berisi minimal dua tabel berelasi sesuai ERD (PK/FK/CHECK)
☐	buat_database.py dan uji_aturan.py berjalan tanpa error
☐	database.py berisi tambah, daftar, ubah, dan hapus dengan query berparameter ?
☐	PRAGMA foreign_keys = ON ada di setiap koneksi
☐	app.py berjalan dengan python -m streamlit run app.py; keempat operasi CRUD bekerja dan input salah menampilkan pesan
☐	Tabel A4 dan B4 terisi hasil asli, tangkapan layar skenario 1, 4a, 6, beserta refleksi